Mini Project 1.2: Measurement-Based WCET Estimation and Confidence Tool

1. The tool

wcet_tool.py is a reusable harness that assumes nothing about the target function. It takes any function and an input generator and follows these steps:

- **Measurement:** each call is timed individually with time.perf_counter() after 20 warm-up calls, and every sample is kept (measure, campaign). Each strategy uses 1000 calls.
- **Statistics:** mean, standard deviation, minimum, maximum and the 90th, 95th and 99th percentiles (statistics). An outlier flag is raised when the maximum exceeds 1.5 times the 99th percentile, which signals a rare spike worth investigating.
- **Safety margin:** the recommended WCET-for-scheduling is the overall observed maximum times (1 + margin). By default the margin comes from a 65% utilization budget, 1/0.65 − 1 = +54%, following the 60–70% convention of Section 1.7. margin_pct overrides it.
- **Strategies and output:** every target has a random and a hand-crafted adversarial strategy. The tool draws a histogram marking each strategy's maximum and the margined estimate, and always prints its limitation statement with the number.

2. Target functions and adversarial inputs

Function (control-flow shape)	Random strategy	Hand-derived adversarial input
early_exit_search (early-exit branch)	Key present at a random position in a 5000-element list (about half a scan on average).	Key absent, so the loop scans all 5000 elements (longest path).
collatz_steps (data-dependent loop count)	n uniform in [1, 100000).	n = 77031, the longest Collatz chain below 100000 (350 steps, confirmed by exhaustive search).
classify_batch (nested conditionals)	400 random integers in [−1000, 1000), mixing cheap and heavy branches.	Every value is non-negative, even and divisible by 3, so all 400 values take the 20-iteration branch.
fir_filter (fixed control flow)	Gaussian noise samples.	Extreme values (10^6). No path change is expected, so this tests for data-dependent timing.

3. Results

Function	Random max (ms)	Adversarial max (ms)	Adv / random	Largest sample from	Margined estimate (ms)	Random max × (1+margin) covers adversarial?
early_exit_search	0.229	0.710	3.10×	adversarial	1.092	no
collatz_steps	0.038	0.091	2.41×	adversarial	0.141	no
classify_batch	0.246	0.839	3.41×	adversarial	1.291	no
fir_filter	0.289	0.552	1.91×	adversarial	0.850	no

Table 1: Random versus adversarial versus margined estimates (margin +54%).

Function	Strategy	Mean (ms)	p99 (ms)	Max (ms)	Max / p99	Outlier flag
early_exit_search	random	0.067	0.168	0.229	1.36	no
early_exit_search	adversarial	0.138	0.213	0.710	3.34	YES
collatz_steps	random	0.006	0.018	0.038	2.07	YES
collatz_steps	adversarial	0.023	0.050	0.091	1.83	YES
classify_batch	random	0.097	0.194	0.246	1.27	no
classify_batch	adversarial	0.315	0.440	0.839	1.91	YES
fir_filter	random	0.070	0.116	0.289	2.49	YES
fir_filter	adversarial	0.072	0.120	0.552	4.62	YES

Table 2: Distribution statistics per strategy. Full statistics are in results_1_2.pdf.

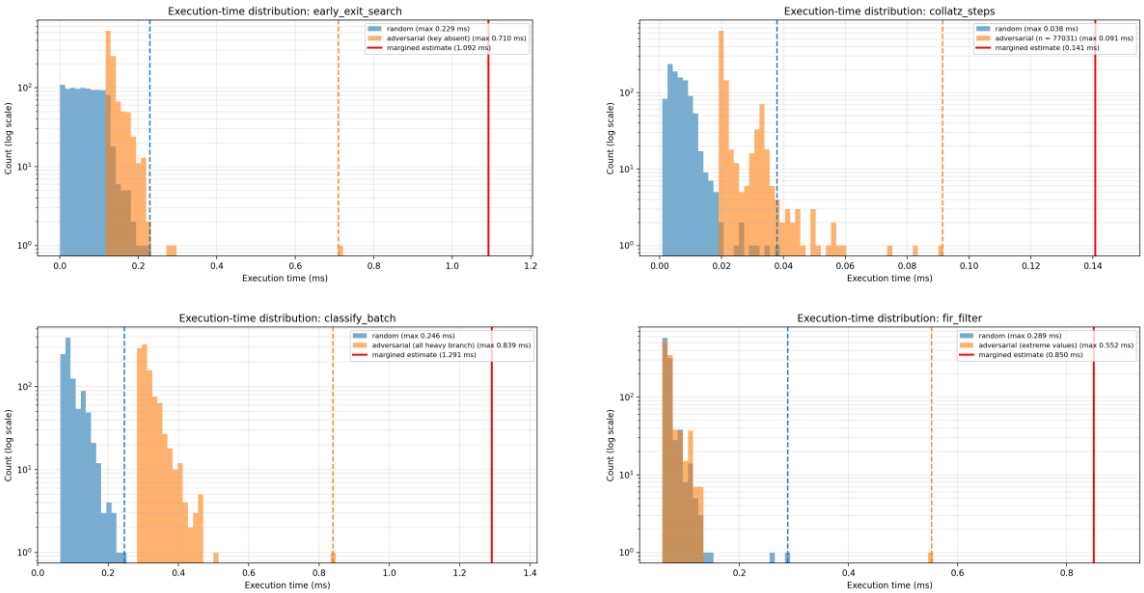


Figure 1: Execution-time histograms per function (dashed = each strategy's measured maximum, red = margined estimate, log-count axis).

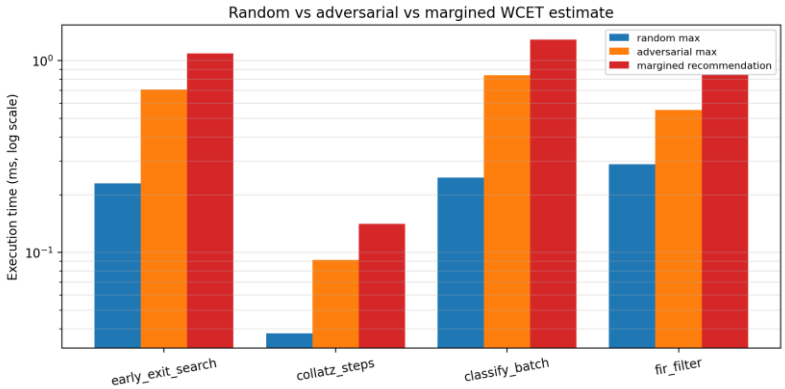


Figure 2: Random maximum, adversarial maximum and margined recommendation per function (log scale).

4. Discussion

Soundness versus pessimism (Section 1.7). Measurement-based estimation is cheap and tight for the paths that were exercised, but it is unsafe: it can only report the slowest run that was observed. Static analysis is sound because it considers all paths and worst-case hardware states, but it is often pessimistic. This tool sits on the unsafe side: it gives a lower bound on the true WCET (for the sampled hardware state), then adds a heuristic margin.

Where the tool was confident and where it was not. For fir_filter, both strategies have the same mean (0.070 ms and 0.072 ms), so the code path does not depend on the data. What remains is timing noise, and the +54% margin is mostly absorbing that noise. Its maxima are 2.5 to 4.6 times the 99th percentile, so the outlier flag fires, and the "adversarial" maximum is an interpreter or OS spike, not a path effect. For early_exit_search the adversarial input is the true worst path, so the estimate for the path is credible. The random strategy alone would have underestimated it by 3.10×. For classify_batch (3.41×) and collatz_steps (2.41×) the random strategy misses the heavy paths because random inputs rarely reach them. The adversarial input only works because I derived it by reading the code, which a tool that assumes nothing about the function cannot do. For an unfamiliar function the tool would underestimate silently. The margin does not repair this: in every function the random-only maximum times (1 + margin) is still below the adversarial maximum (last column of Table 1), so a margin cannot replace path coverage.

Why certification would not accept this output alone (Section 1.8). Certification asks for a bound with an argument that it is safe. A sample of 1000 runs gives no coverage guarantee over paths, cache and pipeline states, interrupts or other load. The platform (interpreted CPython on a general-purpose OS, with garbage collection and frequency scaling) adds noise that cannot be bounded, and the margin is a rule of thumb rather than a proof. The output is useful for early budgeting and as a sanity check that a static bound is not below observed times, but it cannot be submitted as the WCET.

Limitations. Maxima of 1000 samples vary from run to run. In an earlier run the largest fir_filter sample came from the random strategy, and in the final run it came from the adversarial one, which supports the reading that it is noise. The results depend on this machine and run and should be re-measured on the target platform.